

LoadLead — Platform-Wide E2E & Business-Logic Audit

Date: 2026-07-03 **Author:** Platform Engineering **Scope:** full stack (backend + frontend + infra + live prod) **Branch/commit audited:** `main @ 62176f7` **Prod version:** `loadlead-backend-20260703130040`

1. Executive Summary

The platform is **healthy in production and sound at its core**. The two-week feature wave (negotiation e-sign, payments/financing v3, compliance oversight, IAM, beta) is live, the frontend is fully green, and the highest-stakes business logic — the negotiation state machine and the money layer — is well-engineered (conditional writes for concurrency, integer-cents throughout, the Load model never mutated).

However, this audit surfaced **one HIGH-severity environment-parity defect that would cause cross-environment data contamination the moment a staging box is stood up**, plus a non-atomic seam in negotiation assignment and a regression in ADMIN-MFA test coverage. None of these currently break production, but each is a genuine blind-spot.

Severity	Count	Headline
 HIGH	1	Non-prod IaC is stale & unsafe — staging would read/write prod tables for negotiations, compliance, and parts of payments
 MEDIUM	4	Non-atomic accept→assign seam · ADMIN-MFA now untested · negotiation full-table scans · static AWS keys in EB
 LOW	5	Beta gate fails-closed all logins on DDB blip · orphan tables · stale error-handler test · no bid ceiling · config drift
 GREEN	—	Prod hardening, negotiation core, money layer, frontend E2E all verified healthy

Verdict: Safe to keep operating prod. **Do not stand up staging/dev from the current Terraform** until H1 is fixed. Address M1/M2 before the next feature push.

2. Method & Evidence

Surface	What was run	Result
Backend unit/integration	<code>vitest run</code> — 78 files, 620 tests	616 pass / 4 fail (all 4 pre-existing stale tests; not regressions)
Frontend build	<code>vite build</code>	 1,812 modules, 1.86 s
Frontend typecheck	<code>tsc -p tsconfig.app.json --noEmit</code>	 clean
Frontend E2E	Playwright — hauler/shipper/tour specs	 15 / 15 (incl. e-sign gate, 20-min window expiry, both personas)
Live prod smoke	read-only <code>curl</code> probes	 hardened (details §5)
Business logic	manual audit: negotiation, money, beta gate	3 findings (§4)
Environment parity	AWS EB + DynamoDB inspection vs Terraform vs <code>environment.ts</code>	1 HIGH + supporting (§4/§6)

No production data was mutated. All prod interaction was read-only (`GET` /config-describe).

3. Findings — severity-ranked

 **H1 — Non-prod Terraform is stale and would contaminate prod data**

What. The last two weeks of feature tables were added to **prod** (tables created + a handful of EB env vars set by hand), but the **shared Terraform was not kept in sync**:

- The `dynamodb_tableset` module is **missing all three negotiation tables** — `LoadNegotiations`, `NegotiationOffers`, `NegotiationLocks`. A `terraform apply` for dev/staging would **not create them at all**.
- `envs/staging/main.tf` sets **20** `DYNAMODB_*_TABLE` overrides (prefixed `LoadLead-Staging-`), but `environment.ts` defines **39** table slots. The **19** `unset` slots — including negotiations, `AccessorialCharges`, `AdminAuditLog`, `ComplianceGrants`, `StopEvents`, `FactoringAssignments`, `ShipperAgreements`, `PlatformFeePolicy` — **fall back to the hardcoded `LoadLead_*` defaults**, which are the **prod table names**.

Evidence.

```
tableset module:      x MISSING LoadNegotiations / NegotiationOffers / NegotiationLocks
staging env overrides: 20 | environment.ts table slots: 39 | prod EB overrides: 9
staging main.tf:      x MISSING DYNAMODB_{LOAD_NEGOTIATIONS,ACCESSORIAL_CHARGES,ADMIN_AUDIT_LOG,
                        COMPLIANCE_GRANTS,STOP_EVENTS,FACTORING_ASSIGNMENTS,SHIPPER_AGREEMENTS,PLATFORM_FEE_POLICY}_TABLE
prod NEGOTIATION table overrides: 0 (prod works only because default == prod name)
```

Why it's dangerous. Prod is safe today purely because its defaults (`LoadLead_*`) equal its real table names. The instant a staging environment boots with the current config, its backend would keep negotiating, adjudicating disputes, writing the admin audit log, and computing accessorial/factoring charges **against the production tables** — because those subsystems never got a `LoadLead-Staging-` override. That is silent cross-environment write contamination on the money and compliance paths.

Secondary impact. Prod is **not reproducible from IaC**: the tableset module can't recreate the negotiation subsystem, so DR / a clean rebuild would be incomplete. Config for 30 of 39 tables lives implicitly in code defaults, not in the environment.

COA.

1. **(P0, before any staging boot)** Add the 3 negotiation tables to `modules/dynamodb_tableset/main.tf` (keys: `LoadNegotiations` /loadId+GSI, `NegotiationOffers` /negOfferId, `NegotiationLocks` /loadId).
2. **(P0)** Add the missing ~19 `DYNAMODB_*_TABLE` overrides to `envs/staging/main.tf` and `envs/dev/main.tf` so **every** slot in `environment.ts` is env-prefixed. Add a CI check that fails when `environment.ts` gains a table slot with no matching override in each env stack.
3. **(P1)** Add a **fail-closed startup assertion**: under `APP_ENV != production`, refuse to boot if any resolved table name lacks the env prefix (prevents "accidental prod-name fallback" structurally).
4. **(P1)** Reconcile prod: import the ad-hoc tables into the tableset module so `terraform plan` on prod is a clean no-op and prod is reproducible.

🟡 M1 — Negotiation accept→assign is non-atomic; a failure orphans the load

What. `NegotiationService.finishAccepted()` does, in order: (1) conditional transition to `ACCEPTED`, (2) append offer row, (3) `LoadService.assignDriver()`, (4) `releaseLock()`. Steps 1 and 3 touch **different tables** with no transaction. If `assignDriver()` throws (DynamoDB throttle/timeout), the negotiation is already `ACCEPTED`, **the lock is never released** (step 4 unreachable), and the load is left **assigned-to-nobody but hidden from the pool**.

Evidence. `services/negotiationService.ts:449-478`. The idempotent early-returns (`acceptLoad:240`, `acceptOffer:314`) short-circuit on `status === 'ACCEPTED'` **before** re-driving assignment — so a client retry returns "success" while the load stays orphaned. The sweeper (`expireOverdue`) only touches `ACTIVE` statuses, so it never heals an `ACCEPTED` -but-unassigned load. Manual ops intervention would be required.

Impact. Narrow trigger window, but the failure mode is a stuck load that looks accepted to both parties yet is unassigned and un-rebroadcastable. On the money path (agreed rate is bound to the negotiation) this also means an accepted rate with no delivering driver.

COA.

1. Make the idempotent-return paths **reconcile side effects**: before returning an already- `ACCEPTED` negotiation, verify `load.assignedDriverId === haulerDriverId`; if not, re-run `assignDriver` + `releaseLock` (idempotent).
2. Or wrap assignment in `TransactWriteItems` spanning the neg terminal write + the load assignment + lock delete (DynamoDB transactions support multiple tables).

3. Move `releaseLock` into a `finally` so the lock never survives a mid-accept throw.
4. Extend the sweeper to detect `ACCEPTED` negotiations whose load is unassigned and heal them.

● M2 — ADMIN "MFA mandatory" control is live in prod but no longer tested

What. Three `adminMfa` tests fail — not because MFA broke, but because the `beta-gate middleware now sits in front of /login` (`routes/auth.ts:115`) and, in the test harness, its own `Database.query` (the email→role lookup) is un-mocked and **fails closed to `BETA_REQUIRED` (403)** before the handler's MFA branch runs. The suite therefore no longer proves the "reject ADMIN login without 2FA / issue ticket with 2FA" control.

Evidence.

```
adminMfa: expected 'MFA_REQUIRED' → received 'BETA_REQUIRED' (tests never reach the MFA branch)
routes/auth.ts:115 requireBetaGate({mode:'login'}) ← runs before the MFA check at :128
middleware/betaGate.ts:133-137 catch → fail-closed → rejectAsBeta
```

Impact. Prod behavior is fine (a real ADMIN row exists, so the gate exempts it — `betaGate.ts:124` — and MFA is enforced). The problem is **coverage regression**: a future change that silently disables ADMIN MFA would ship green. Four red tests in CI also erodes the "all green" signal.

COA. In the `adminMfa` (and `errorHandler`) tests, either stub the `beta-gate Database.query` to return the ADMIN user, or set `BETA_MODE=off` for that suite, so the MFA branch is actually exercised. Then re-assert the 3 MFA behaviors. Target: **620/620 green**.

● M3 — Negotiation reads are full-table scans; long-poll multiplies them

What. `latestForLoad` , `offersFor` , and `activeLockedLoadIds` use `Database.scan` (whole-table), filtering by `loadId` in memory. The long-poll `waitForChange` calls `latestForLoad` **every ~1 s for up to 25 s per client** — i.e. up to 25 full scans of the negotiations table per watcher per poll cycle.

Evidence. `services/negotiationService.ts:114-122, 218-231, 407-419` . The negotiation tables are absent from the tableset module, so they carry **no GSI** to query by `loadId` .

Impact. Fine at beta volume; $O(\text{table})$ per action and per poll tick. With concurrent live negotiations and multiple watchers this is a cost and latency cliff (DynamoDB scan RCU scales with table size, not result size).

COA. Add a `loadId-index` GSI to `LoadNegotiations` / `NegotiationOffers` and a direct `GetItem` on `NegotiationLocks(loadId)` ; replace the scans with queries. (Fold into the H1 tableset work.)

● M4 — Static AWS credentials in the Elastic Beanstalk environment

What. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are set as EB application env vars rather than relying on the EC2 **instance profile**.

Impact. Long-lived static keys carry rotation burden and a broader exposure surface (anyone with EB config read sees they exist; a leaked instance dump exposes usable keys). Instance-profile role credentials are short-lived and auto-rotated.

COA. Grant the EB EC2 instance profile the needed DynamoDB/S3/SES permissions, remove the two env vars, and let the SDK use the instance role. Rotate the existing keys after cutover.

● LOW findings

#	Finding	Evidence	COA
L1	Beta gate fails closed for all logins (incl. ADMIN) on any users-table error, showing the neutral "private beta" message. A DDB blip locks everyone out of web login; the bootstrap CLI is the only escape.	<code>betaGate.ts:133-137</code>	Let genuine infra errors surface as 503 (retryable) rather than masquerading as <code>BETA_REQUIRED</code> ; keep fail-closed only for "not allowlisted".
L2	Orphan / duplicate tables: <code>LoadLead</code> , <code>loadlead</code> , <code>LoadLead_AdminAudit</code> (vs the used <code>_AdminAuditLog</code>), <code>LoadLead-MembershipAuditLogs</code> (dash form).	<code>dynamodb list-tables</code> (56 total)	Confirm unused, snapshot, delete. Removes cost + wrong-table risk.
L3	<code>errorHandlerDoubleResponse</code> test asserts stack-in-body keyed on <code>NODE_ENV</code> , but the handler keys off <code>APP_ENV</code> (the deliberate split). Stale test.	vitest fail <code>[1/4]</code> ; <code>environment.ts:10-15</code>	Update the test to toggle <code>APP_ENV</code> .
L4	No upper bound on negotiation bid/counter rate (<code>validateAmount</code> floors at 1 cent, no ceiling).	<code>negotiationService.ts:487-507</code>	Add a sane per-mile / total ceiling to reject fat-finger outliers.
L5	Config-centralization drift: <code>FactoringOptIns</code> / <code>CarrierFactoringProfiles</code> are read via direct <code>process.env</code> while 39 other tables go through the central <code>config</code> object.	prod EB has <code>DYNAMODB_FACTORING_*</code> keys absent from <code>environment.ts</code>	Route all table names through <code>config.dynamodb.*</code> for one source of truth.

4. What's healthy (verified GREEN)

- **Prod hardening.** `/api/health` → `productionHardened:true`; test routes (`/api/_test/*`) return **404** (pruned from the bundle, verified at runtime); Helmet headers present (**CSP, HSTS preload, X-Frame DENY, nosniff**); CORS locked to the three real domains; `JWT_SECRET` set (not the `dev-secret` fallback); `ALLOW_ADMIN_BOOTSTRAP=false`; `NODE_ENV=production`; `APP_ENV=production`.
- **Negotiation core.** One-active-negotiation-per-load via conditional `attribute_not_exists` lock; turn-taking + terminal transitions all use conditional writes (no double-apply); accepts are idempotent; **unlimited rounds within a hard 20-minute window** (lazy `expireIfOverdue` + swept `expireOverdue`), rebroadcast at the original posted rate; the **Load row is never mutated**; agreed rate is bound to the negotiation for settlement.
- **Money layer.** Integer cents everywhere; deterministic half-up rounding; `Number.isSafeInteger` guards; a single sanctioned dollars→cents boundary for the legacy Load model. (`utils/money.ts`)
- **Frontend.** Build + typecheck clean; **15/15** Playwright E2E including the e-sign consent gate (H8), the 20-minute window expiry via long-poll (H9), and full shipper/hauler negotiation flows.
- **Backend suite.** 616/620 green; the 4 failures are the known stale-test set (\$M2, \$L3), confirmed **not** regressions.

5. Live production smoke (read-only)

```

GET /api/health 200 {ok:true, productionHardened:true}
GET /api/_test/outbox 404 (test routes not in prod bundle) ✓
GET /api/_test/reset 404 ✓
GET /api/negotiations/x 401 (auth gate) ✓
POST /api/negotiations/x/accept-load 401 (auth gate) ✓
GET /api/admin/compliance/me 401 ✓
GET /api/owner-operator/dashboard 401 ✓
Headers: CSP · HSTS(preload) · X-Frame-Options:DENY · X-Content-Type-Options:nosniff ✓
Frontend https://loadleadapp.com/ 200 (serving new bundle) ✓
Flags: ALLOW_ADMIN_BOOTSTRAP=false · BETA_MODE=true · CORS=3 locked domains ✓

```

Note: `GET /api/loads` returns 404 (no unauth collection route) — expected, not a defect.

6. Environment parity matrix

Dimension	dev	staging	prod	Discrepancy
EB environment running	✗ none	✗ none	✓ loadlead-backend-prod	Only prod exists → no pre-prod test bed; parity is aspirational
Terraform stack defined	✓	✓	✓	—
Negotiation tables in IaC	✗	✗	⚠ ad-hoc	Missing from dynamodb_tableset (H1)
Table env-prefix coverage	partial	20/39	9/39 (defaults)	staging half-overridden → prod-name fallback (H1)
Table isolation guarantee	✗	✗	n/a	staging would hit prod tables for negotiations/compliance/payments
Frontend env file	none	none	.env.production	No .env.development ; local relies on Vite defaults

Bottom line: there is currently only one real environment (prod). The IaC that would create the others is stale and, if applied as-is, unsafe. "Parity" cannot be claimed until H1 is closed and a staging box is actually stood up and smoke-tested.

7. Courses of Action — prioritized

Pri	Action	Finding	Effort
P0	Add negotiation tables to the tableset module + complete the staging/dev env-var override maps (all 39 slots)	H1	~½ day
P0	Add a non-prod boot assertion: refuse to start if any table name lacks the env prefix	H1	~2 h
P1	Make accept→assign idempotent+reconciling (or transactional); lock-release in finally ; sweeper heals ACCEPTED-unassigned	M1	~½ day
P1	Fix adminMfa + errorHandler tests to bypass/stub the beta gate → 620/620 green	M2, L3	~2 h
P1	Move to EB instance-profile creds; remove static keys; rotate	M4	~2 h
P2	Add loadId GSI + convert negotiation scans to queries	M3	~½ day
P2	CI guard: environment.ts table slot ↔ every env stack override must match	H1	~2 h
P3	Beta-gate: surface infra errors as 503, not BETA_REQUIRED	L1	~1 h
P3	Delete orphan tables (after snapshot); add bid ceiling; centralize factoring table config	L2,L4,L5	~2 h

8. Recommendations (strategic)

- Stand up a real staging environment** immediately after H1 — the platform has run two weeks of money/compliance features with **no pre-prod test bed**. Every deploy is currently prod-first. A hermetic staging with its own prefixed tables is the single highest-leverage reliability investment.
- Treat IaC drift as a release gate.** The root cause of H1 is that prod got hand-applied tables/env vars while the Terraform lagged. Make "tables + env overrides present in every env stack" a required CI check so this can't recur silently.
- Add cross-table transactional integrity to the assignment chokepoint.** M1 is the template for any place a terminal state in one table drives a write in another (assignment, settlement, factoring advance). Audit those seams next.
- Guard the security-control coverage.** M2 shows a security test can rot into a false-green. Add a smoke test that asserts ADMIN login without MFA is refused, run against a real (staging) environment, not just mocked units.
- Next audit pass:** deep-dive the **payments ledger + settlement take-rate** and the **compliance intercept/suppression seams** — this pass sampled them (their unit suites are green) but did not trace them end-to-end the way negotiation was traced.

Appendix — commands and raw outputs are reproducible from the audit session; all prod interaction was read-only.